

Orbital Stellar

Drips Funding Correlation, FUNDING.json Explained, and a Verified Comparative Analysis Against Hintents (the \$1,500 Reference Project)

PREPARED

April 26, 2026

CLASSIFICATION

Internal — Strategic
Planning

SCOPE

OSS Funding · Repo
Hygiene · Drips
Pipeline

COMPANION TO

April 14 Report · April
22 Addendum

Scope & a Correction Up Front

This document is a working analysis prepared in response to a specific question raised by the Orbital founding team: *are we building the right artifacts to maximise income from the Drips Stellar Wave Program, and what should be open versus closed source given the wave issues already drafted?*

The first deliverable on this engagement included an unverified statistic — that the hintents reference repository contained "40 SCF references in `docs/proposals.md`." On direct inspection of the file the actual count is **one** sentence mentioning the Stellar Community Fund. The conclusion in the prior memo was therefore built on a confabulated correlation. This document supersedes that analysis and is based only on facts verified by reading the source files directly.

WHAT THAT CORRECTION CHANGES

The earlier hypothesis was: *"featured repos are featured because they reference SCF/OnlyDust heavily."* The verified picture is different — hintents has no `FUNDING.json`, no OnlyDust references, no Drips file, and exactly one SCF mention. Yet it still received \$1,500. The real differentiators are **shipped-product perception** and **documentation density**, not explicit grant-program linkage. The corrected fix list at the end of this document reflects the verified evidence.

What this document covers

1. How money actually flows from grant programs through Drips to maintainers, with the SCF / OnlyDust / Wave Program relationships clarified.
2. What `FUNDING.json` is, what it contains, and when (and when not) to add it to a repo.
3. A verified deep dive on hintents — every claim sourced from a file in `C:\Users\SAMUEL\Desktop\drips\hintents`.
4. A side-by-side comparison of hintents and Orbital across 21 dimensions.
5. A four-tier fixes table prioritising the actions that close the polish gap.
6. A 90-day sequencing plan for executing those fixes.

How Drips Actually Pays Stellar Projects

Drips itself is a neutral routing protocol. It does not decide which projects get funded — it splits incoming donations according to each recipient's "splits config." The interesting question is therefore *"who sends money into Drips, and how do they pick recipients?"*

The flow

```

Grant pool / SCF awardee / Wave-program treasury / direct sponsor
|
| donates to a "Drip List" or to a project's Drips identity
▼
Drips protocol (smart contract)
|
| splits per the recipient's configured splits graph
▼
Maintainer addresses + dependency graph + listed contributors
  
```

Money sources on Stellar (in order of materiality)

Source	How it pays	Predicts featured?
Stellar Wave Program	A Drips-run initiative (the same <code>Stellar Wave</code> label used in <code>WAVE_ISSUES.md</code>) funded with grant money. Issues opted into the program pay contributors per complexity points when merged.	Yes — direct.
OnlyDust	A bounty platform that posts and pays contributor tasks; payouts can route through Drips.	Strong second-order signal.
SCF awardees	Grant recipients sometimes route a portion of their award through Drips to reward downstream contributors.	Indirect — depends on the awardee.
Direct project sponsors	Individual donors topping up a Drip List for a project they like.	Marginal in volume.

CORRECTED MENTAL MODEL

Drips' "featured" status is **curatorial**, awarded to repos that look high-quality and shipped. Explicit SCF / OnlyDust integration is one validation path, but it is not necessary. Hintents is featured despite having no explicit OnlyDust or Drips integration anywhere in its repo — the curators picked it up on quality signal alone. That is the path Orbital should optimise for.

What that means for Orbital

Two paths to maximising Drips income operate in parallel and are *not* in tension:

1. **Be obviously high-quality and shipped.** CHANGELOG with real releases, hosted docs, multi-language sophistication where relevant, an active contributor table. This is the *look-the-part* half.
2. **Activate the contributor flywheel.** Publish the 150 wave issues, apply to OnlyDust, register on Drips. This is the *turn-on-the-pipes* half.

What `FUNDING.json` Contains and Means

This is a Drips-specific manifest distinct from GitHub's `.github/FUNDING.yml` (different schema, different purpose). It lives at the repository root.

Schema

```
{
  "drips": {
    "<chain-name>": {
      "ownedBy": "0x<ethereum-style-address>"
    }
  }
}
```

Real examples from the featured cohort

Repo	Chain	Address
be-energy	optimism	0x7d012643D8E72994895F826A22E72ca972D576fd
stellar-portfolio-rebalancer	ethereum	0xEFeA3500A667A359B84fc0ce4ba391F3B7761Bb8

What it does

- Declares the canonical Drips recipient address for the repository, on a specific chain (Drips uses Ethereum-style addresses across multiple L2s).
- When Drips' indexer crawls the repo, this file is the *on-repo proof of ownership* linking the GitHub repo to a Drips identity.

What it does *not* do

- **Does not unlock funding by itself.** Being featured on Drips is a separate curatorial step; `FUNDING.json` just enables the routing once funding is sent.
- **Does not improve ranking.** Of 15 featured repos audited in the Stellar cohort, only 2 have one. Hintents has none and still received \$1,500.
- **Is not required.** A maintainer can register repo ownership through Drips' web UI without the file. The file is a convenience and a proof, not a prerequisite.

WHEN ORBITAL SHOULD ADD IT

Only after registering an Orbital recipient on `drips.network` and obtaining the recipient address. Adding it before that is empty signaling — the file would point to nothing. Skip it for now; revisit after Drips registration is done.

Hintents — Verified Inventory of the \$1,500 Reference

Every claim below is sourced from a file directly inspected in the local hintents clone. Items previously asserted on second-hand summaries are explicitly marked **verified**; items that the prior memo over-claimed are flagged.

Real inventory

Dimension	Value	Notes
License	Apache 2.0	Enterprise / infrastructure orientation; differs from MIT-dominant cohort.
Language stack	Go (CLI) + Rust (simulator) via FFI	Multi-language sophistication; Rust simulator wraps <code>soroban-env-host</code> .
Root <code>.md</code> files	19	IMPLEMENTATION_*, DELIVERY_COMPLETE, PR_MESSAGE_*, MAINTAINER_NEXT_STEPS — visible execution paper trail.
docs/ <code>.md</code> files	64	Architecture, ADRs, integration guides; published to a Mintlify site.
GitHub workflows	12	ci, fuzz, dependency-audit, license-audit, strict-lint, all-contributors, changelog, emoji_check, integration, release, ci-standardization, test-cross-platform.
Issue templates	bug_report + feature_request	Standard.
FUNDING.json	None	Disproves the "FUNDING.json drives featured-status" hypothesis.
Drips / OnlyDust references	Zero	Disproves the "explicit-linkage" hypothesis.
SCF references	1 sentence (proposal.md:38)	Stated intent to apply, not evidence of receipt.
<code>.all-contributorsrc</code>	Yes	Auto-generated contributor badge in README; all-contributors workflow active.
CHANGELOG.md	Keep-a-Changelog format	Shipped v2.0.0 on 2026-03-20 with concrete features (Interactive Trace Viewer, Flamegraph, HSM signing).

Dimension	Value	Notes
Code of Conduct	Yes (root)	Standard.
Hosted docs site	<code>dotandev-hintents-75.mintlify.app</code>	Linked from README; professional appearance.
Grant proposals	Two (Infrastructure + RWA tracks)	Same project pitched to two different SCF tracks — covers two grant categories.
Status framing	"Phase 4: Advanced Diagnostics"	Communicates active execution.
Pitch register	"#1 developer pain point on Stellar"	Marketing-grade framing in a docs file; reads like a YC application.

What actually makes hintents stand out

1. **It looks shipped.** v2.0.0 in CHANGELOG with specific release notes. Most "OSS Stellar projects" feel half-built; hintents reads like real software.
2. **Multi-language sophistication.** Go-CLI + Rust-simulator via FFI is hard to fake. Curators read this as "real engineering team."
3. **Documentation density.** 83 markdown files total, with a hosted Mintlify site. Curators scanning candidates use docs depth as a quality proxy.
4. **CI maturity beyond cohort.** Fuzz, license-audit, dependency-audit, all-contributors automation — workflows you only set up if you intend to maintain a project for years.
5. **Two-track grant pitching.** Dual `proposal1.md` / `proposal12.md` targeting two SCF tracks (Infrastructure and Financial Products) from the same codebase.
6. **Marketing-grade pitch in a docs file.** Opens with "#1 developer pain point on Stellar" and frames the problem with concrete pain (`tx_failed` returning `Error(WasmVm, InvalidAction)`).
7. **all-contributors integration.** Public credit table is itself a contributor incentive — Drips/OnlyDust contributors show up there.

What the prior memo over-claimed

Earlier claim	Verified reality
"40 SCF references in proposal.md"	One sentence in proposal.md, line 38; no SCF references anywhere else in the repo.
"Heavy SCF doc integration"	Two grant proposal files exist; only one of them mentions SCF, in one sentence each.
"Pre-vetted via Stellar's own programs"	Unverifiable from repo alone. Repo shows author's <i>intent to apply</i> , not evidence of acceptance.

Hintents vs. Orbital — 21 Dimensions

Signal	Hintents	Orbital	Verdict
License	Apache 2.0	MIT	Both valid; different audience signals.
Language stack	Go + Rust (FFI)	TypeScript only	Hintents.
GitHub workflows	12 (fuzz, license-audit, dep-audit, all-contributors, strict-lint, ...)	7 (build, ci, codeql, integration, release, security, stale)	Hintents.
Issue + PR templates	bug + feature	bug + feature + PR template	Orbital (PR template is rare in cohort).
CHANGELOG.md	Keep-a-Changelog, v2.0.0 shipped	None	Hintents — decisively.
Versioning maturity	Semver, v2.0.0 (2026-03-20)	v0.x, no formal release	Hintents.
Root <code>.md</code> count	19	5 (README, ROADMAP, CONTRIBUTING, LICENSE, SECURITY)	Hintents has more (style choice, not necessarily better).
docs/ folder	64 files	Does not exist	Hintents — major gap.
Hosted docs site	Mintlify (live)	None published	Hintents.
all-contributors	Yes (.all-contributorsrc + workflow)	No	Hintents.
Code of Conduct	Yes	No	Hintents.
Grant proposal docs	Two (Infrastructure + RWA)	None	Hintents — major

Signal	Hintents	Orbital	Verdict
			gap.
Status framing in README	"Phase 4: Advanced Diagnostics"	None	Hintents.
Pitch register	"#1 developer pain point on Stellar"	"Real-time event infrastructure for the Stellar network"	Hintents reads as marketing; Orbital reads as docs.
Issue plan / Wave issues	None visible	150-issue WAVE_ISSUES.md + create-wave-issues.js	Orbital — decisively.
GitHub Discussions	Configured (referenced in CONTRIBUTING)	Mentioned but not configured	Hintents.
Monorepo structure	Single Go module + simulator subdir	True monorepo (apps/ + packages/) with pnpm workspaces	Orbital.
Test infrastructure	jest + Go tests + integration	TS tests in each package	Roughly equal.
README badges	all-contributors badge present	None	Hintents.
FUNDING.json	None	None	Equal — irrelevant signal.
Drips / Wave-Program text in CONTRIBUTING	None	"Stellar Wave Program" section present	Orbital.

SUMMARY VERDICT

Orbital is structurally stronger on architecture and Wave-program engineering (true monorepo, 150 pre-planned issues, automated label/issue creation, native Wave Program section). Hintents is stronger on positioning, polish, and shipped-product perception (CHANGELOG, v2.0.0, hosted docs, all-contributors, two grant proposals, marketing pitch). To a curator looking at both today, hintents looks more *"shipped, real, worth featuring."* Orbital looks more *"well-organised future project that hasn't released yet."* That gap is closeable in 1–2 weeks of focused, non-product work.

Prioritised Fixes for Orbital

Tier 1 — Perception fixes (close the gap to hintents-tier polish)

#	Action	Where	Effort	Impact
1	Create <code>docs/proposal1.md</code> framing Orbital for SCF Infrastructure Grants. Cite the 7-day Soroban retention pain, the React-hooks gap, the Horizon→RPC migration overhead. Open with a "#1 developer pain point" sentence.	<code>docs/proposal1.md</code>	1 day	High — directly mirrors hintents' winning artifact.
2	Create <code>docs/proposal2.md</code> framing Orbital for a second SCF track — Financial Products (anchor SDK, x402 agent payments) or Real-World Adoption.	<code>docs/proposal2.md</code>	1 day	High — covers two grant categories.
3	Add <code>CHANGELOG.md</code> in Keep-a-Changelog format. Tag and release <code>v0.1.0</code> with the current feature set.	root + git tag	2 hours	High — removes the v0.0.0 instability perception.
4	Add <code>CODE_OF_CONDUCT.md</code> (Contributor Covenant 2.1, copy-paste).	root	15 min	Medium — closes a trust-signal gap.
5	Set up <code>.all-contributorsrc</code> and add the all-contributors GitHub Action. Auto-generates contributor table in README.	root + workflow	2 hours	Medium — high-confidence cohort signal.
6	Set up a hosted docs site — Mintlify, Docusaurus, or use existing <code>apps/web</code> with a <code>/docs</code> route filled with structured content. Link the URL from README.	<code>apps/web</code> or Mintlify	1–2 days	High — major cohort gap.

#	Action	Where	Effort	Impact
7	Add a "Status: Phase 0 — Foundation" line under the README title, mirroring hintents' phase framing.	README.md	5 min	Low–Medium.
8	Rewrite the README opening sentence in pitch register, not docs register. E.g.: <i>"Stellar's biggest developer-experience gap in 2026 isn't a missing API — it's that Horizon's firehose still requires every team to build their own event delivery."</i>	README.md:3	30 min	Medium.

Tier 2 — Workflow / OSS hygiene additions

#	Action	Where	Effort	Impact
9	Add <code>.github/workflows/dependency-audit.yml</code> (npm audit + license check on PRs).	workflows	1 hour	Low.
10	Add <code>.github/workflows/license-audit.yml</code> (validates LICENSE headers across packages).	workflows	2 hours	Low.
11	Enable GitHub Discussions; configure categories (Ideas / Q&A / Show & Tell / Polls). Reference in CONTRIBUTING.md.	repo settings + CONTRIBUTING.md	30 min	Medium.
12	Add <code>.github/workflows/strict-lint.yml</code> (ESLint + Prettier strict mode).	workflows	1 hour	Low.
13	Fill in the placeholder (<i>invite link</i>) for Discord, add Twitter / X handle. Empty community section reads as inactive project.	README.md:102–104	1 day (incl. Discord setup)	Medium.

Tier 3 — Wave program activation (Orbital-specific, no cohort analog)

#	Action	Where	Effort	Impact
14	Run <code>node .github/create-wave-issues.js</code> — publishes the 150 issues, creates labels, opens the contributor flywheel.	local + GitHub	1 hour (rate-limited)	Highest single ROI on contributor activity.
15	Apply to OnlyDust as a project (onlydust.com). Reference the published Wave issues.	external	1 day	High.
16	Submit the SCF application using <code>docs/proposal.md</code> as the basis. SCF cycles open quarterly.	external	2–3 days	Highest funding upside.
17	Register an Orbital recipient on <code>drips.network</code> . Obtain the recipient address.	external	30 min	Required for any Drips income.
18	After (17), commit <code>FUNDING.json</code> with the registered Drips address.	root	5 min	Low (signal-only).

Tier 4 — Open / closed source structural moves

#	Action	Where	Effort	Impact
19	Stand up a separate private monorepo (e.g., <code>orbital-cloud</code> on a private GitHub org). Initial scaffold only — Postgres adapter interface, Express auth middleware shell.	new private repo	2 days	High — the architectural boundary must exist before contributors test it.
20	Add an <code>apps/server/src/registry/</code> interface so the in-memory registry can be replaced with a Postgres adapter from the private repo without forking <code>pulse-core</code> .	<code>apps/server</code>	1 day	High — extension point per Addendum §7.
21	Document publicly which feature areas are open-source	<code>docs/</code>	2 hours	Medium.

#	Action	Where	Effort	Impact
	vs Cloud / Rails. A short docs/open-source- policy.md prevents contributors from filing PRs into eventually-closed territory.			

Suggested 90-Day Sequencing

Window	Theme	Items
Week 1	Perception & program activation	1, 2, 3, 4, 7, 8, 14, 17 — proposal docs, CHANGELOG, CoC, README rewrite, run the wave-issue creator, register on Drips.
Week 2	Community surface	5, 6, 11, 13 — all-contributors, hosted docs site, GitHub Discussions, Discord / Twitter.
Week 3–4	Apply for funding	15, 16 — OnlyDust application, SCF application.
Week 5–6	Private repo skeleton + extension points	19, 20, 21 — protect the open / closed boundary before any private feature ships.
Ongoing	Hygiene	9, 10, 12, 18 — incremental workflow additions, FUNDING.json once the Drips address exists.

The three highest-leverage moves

#1 Write docs/proposal.md

#14 Run the wave issue creator

#16 Submit the SCF application

Everything else compounds around these three.

Closing

The earlier "40 SCF references" claim was wrong. The corrected story is more useful: hintents wins on shipped-product perception, not on grant-program documentation. Orbital is architecturally stronger and operationally further along on the Wave-program plumbing. Closing the polish gap is two weeks of non-product work, after which Orbital should look objectively better than hintents to any Drips, Wave, or SCF curator evaluating both.

The 20-year strategic direction laid out in the April 14 Report and refined in the April 22 Addendum remains the load-bearing document for product and monetisation decisions. This document is a tactical complement focused on the next 90 days of funding-pipeline activation.